

ConANN: C Prototype and Python Package for Adaptive IVF Search

RAJ LOHAR Roll No.: BSD-CC-2414
Bhagavath Chukka Roll No.: BSD-DH-2424
Tanish Kothari Roll No.: BSD-DH-2424
Madhumita Das Roll No.: CS2515
SUMAN Polley Roll No.: CS2535
Course: Database Management System Code: Project 1

June 2026

Abstract

Approximate nearest neighbor search reduces vector search cost by scanning only a part of the database. The Inverted File index, or IVF, is a common way to do this. IVF groups vectors into centroid lists and searches only a selected number of lists for each query. The main control parameter is `nprobe`. A small `nprobe` gives faster search but can miss true nearest neighbors. A large `nprobe` gives better recall but scans more vectors.

The ConANN paper applies conformal risk control to IVF search. Instead of using one fixed probe count for every query, ConANN calibrates a stopping rule from queries with known exact nearest neighbors. At query time, the search probes lists in order and stops when the calibrated score says the requested false negative rate has been reached.

This project had two parts. First, we wrote a standalone C prototype that implements IVF, calibration, adaptive probing, and a benchmark against FAISS IVFFlat on SIFT-style data. The C prototype controlled false negative rate near the requested targets, but it did not beat FAISS runtime. This result exposed where the paper gets most of its speed: the adaptive rule must run inside the optimized FAISS scan path. Second, we packaged the ConANN-modified FAISS C++ code as a Python package named `conann`. The package builds CPU wheels for Linux and Windows, supports CPython 3.10 through 3.14, and exposes `calibrate_conann` and `search_conann` through Python.

Project Repositories

- Python package: <https://github.com/smoothy100x/conann/>
- C benchmark prototype: <https://github.com/smoothy100x/conann-c-benchmark>
- Our Implementation in C (with Fuzzy K-Means): https://github.com/shadow23-cmi/Fuzzy_Kmeans_ConANN

Contents

1	Introduction	4
2	Related Work	4
2.1	FAISS and IVF	4
2.2	ConANN	4
2.3	Conformal Risk Control	5

3	Terminology and Definitions	5
3.1	Nearest Neighbor Search	5
3.2	Recall and False Negative Rate	5
3.3	Inverted File Index	5
4	Theory Behind ConANN	6
4.1	What the Paper Changes	6
4.2	Calibration	6
4.3	Adaptive Query-Time Search	6
4.3.1	Formal Algorithm	7
5	Part 1: C Implementation	7
5.1	Goal of the C Prototype	7
5.2	Code Organization	8
5.3	Data Flow	9
5.4	Adaptive Search in C	9
5.5	Where the C Prototype Differs From the Paper	9
6	Part 1 Results	9
6.1	Benchmark Setup	9
6.2	Measured Results	9
6.3	Additional SIFT10M Result	10
6.4	Additional GIST Result	12
6.5	Interpretation	14
7	Part 2: Python Package	14
7.1	Goal	14
7.2	Python API	15
7.3	Staging the Package	16
7.4	Examples	16
8	Linux Wheel Build	16
8.1	Why Manylinux Was Needed	16
8.2	Linux Build Process	17
8.3	Linux Wheel Matrix	17
9	Windows Wheel Build	18
9.1	Windows Requirements	18
9.2	Native Dependencies	18
9.3	MSVC Fixes	18
9.4	Windows Wheel Matrix	18
10	Verification	19
10.1	C Prototype Verification	19
10.2	Python Package Verification	19
11	Detailed Design Notes	20
11.1	Why the Project Was Split Into Two Parts	20
11.2	C Data Structures	20
11.3	Calibration Cache	20
11.4	Why FNR Control Worked But Runtime Did Not	20

12 Python Package Engineering	21
12.1 Import Name and Namespace	21
12.2 Why Wheels Were Needed	21
12.3 Linux Build Details	21
12.4 Windows Build Details	22
12.5 Metadata Repair	22
13 How a User Would Use the Package	22
13.1 Basic Vector Search	22
13.2 PyTorch Embeddings	23
13.3 Text Vectors	23
14 Comparison With the Paper	23
15 Limitations	24
16 Conclusion	24

1 Introduction

Modern search and recommendation systems often represent text, images, audio, or user behavior as dense numeric vectors. A nearest neighbor query asks for the database vectors closest to a query vector. Exact search compares the query to every vector in the database. That gives the right answer, but the cost grows linearly with the number of vectors. For a database with millions of vectors, exact search can be too slow for interactive use.

Approximate nearest neighbor search reduces this cost by accepting a controlled amount of error. FAISS is one of the most used libraries for this task. One of its standard index families is IVF. IVF first partitions vectors into lists using centroids. During search, the query is compared with the centroids, and the system scans only the nearest lists. The number of scanned lists is usually called `nprobe`.

The difficulty is that one fixed `nprobe` does not match every query. Some queries lie close to the center of one list and need only a few probes. Other queries lie near list boundaries and need more probes. A fixed value wastes work on easy queries or misses true neighbors on hard queries. The ConANN paper by Horchidan et al. addresses this problem by treating IVF search as a risk controlled prediction task [1]. It uses conformal risk control to choose a threshold that controls the expected false negative rate.

The project described in this report followed that idea in two stages. The first stage was a C implementation. We wrote k-means training, inverted lists, fixed IVF search, adaptive ConANN-style search, and benchmark code. That stage made the algorithm visible and easy to inspect. The second stage moved closer to the paper’s implementation path. We built a Python package from the ConANN-modified FAISS fork, compiled native wheels for Linux and Windows, and published the package as `conann`.

2 Related Work

2.1 FAISS and IVF

FAISS is a vector search library developed for fast similarity search on dense vectors [2]. It supports exact indexes, inverted-file indexes, product-quantized indexes, graph indexes, and GPU execution. This project used the CPU IVFFlat path because the ConANN paper also evaluates IVFFlat for the main validity and efficiency experiments.

In IVFFlat, vectors remain in their original floating-point form. The index stores them in inverted lists assigned by a coarse quantizer. Query speed mainly depends on how many lists are scanned and how fast each list is scanned. FAISS has optimized memory layout, distance kernels, OpenMP parallelism, and C++ integration that are hard to reproduce in a small C program.

2.2 ConANN

ConANN adds statistical risk control to IVF search [1]. It does not ask the user to hand tune `nprobe`. It calibrates a threshold for a target false negative rate. During search, it probes the nearest centroid lists in order and stops once the calibrated rule accepts the current result set.

The paper reports that ConANN can meet requested false negative rates while probing fewer clusters than a fixed FAISS baseline in many cases. A useful detail for this project is that the paper implementation integrates into FAISS 1.9.0. That means it can reuse FAISS distance computation, heap handling, list scanning, and memory layout. The standalone C prototype in this project could implement the algorithmic idea, but it did not inherit those optimized internals.

2.3 Conformal Risk Control

Conformal risk control gives finite-sample risk bounds under exchangeability assumptions [3]. In this project, the risk is the false negative rate of approximate nearest neighbor search. Calibration queries are searched exactly to obtain ground truth. Approximate search is then simulated at different probe depths. The calibration step chooses a threshold whose corrected empirical risk is below the target.

This form of calibration fits IVF search because IVF can generate intermediate result sets as more lists are scanned. The search path already has a natural ordering: the closest centroid list is scanned first, then the next closest list, and so on. ConANN attaches a risk-controlled stopping rule to that ordered process.

3 Terminology and Definitions

3.1 Nearest Neighbor Search

Let $X = \{x_1, x_2, \dots, x_n\}$ be a database of d -dimensional vectors. For a query vector q , exact k nearest neighbor search returns the k vectors with the smallest distance to q . This project uses squared Euclidean distance:

$$d(q, x) = \sum_{j=1}^d (q_j - x_j)^2. \quad (1)$$

Exact search evaluates this distance for every database vector. Approximate search evaluates fewer vectors and may miss some true neighbors.

3.2 Recall and False Negative Rate

Let $G(q)$ be the exact set of k nearest neighbor ids for query q , and let $A(q)$ be the approximate set returned by an index. Recall at k is:

$$\text{Recall}(q) = \frac{|A(q) \cap G(q)|}{k}. \quad (2)$$

The false negative rate used in this report is:

$$\text{FNR}(q) = 1 - \text{Recall}(q). \quad (3)$$

For a query set, the measured FNR is the average of per-query false negative rates. A target FNR of 0.10 means the system should miss about 10 percent of the true top- k neighbors on average.

3.3 Inverted File Index

An IVF index trains N centroids with k-means. Each database vector is assigned to the nearest centroid and stored in that centroid's inverted list. During search, the query is compared with the centroids. The index scans only the nearest O lists, where O is the probe depth. FAISS calls this value `nprobe`.

The fixed FAISS baseline chooses one probe value and uses it for every query. ConANN chooses the number of probes per query by applying a calibrated stopping rule.

4 Theory Behind ConANN

4.1 What the Paper Changes

The ConANN paper starts from a simple observation: IVF search already produces a sequence of intermediate answers. After probing one list, the index has a top- k candidate set. After probing two lists, it has another candidate set. This continues until all lists have been probed. ConANN asks whether a calibrated rule can stop this sequence early while keeping average FNR below a target.

For a query q , a requested neighbor count k , and a probe depth O , let $P_O(q)$ be the candidate result set after probing O centroid lists. The paper uses a scalar score based on the distance of the k -th returned neighbor:

$$s(q, O) = d_k(q, P_O(q)). \quad (4)$$

When O grows, the candidate set improves or stays the same. The k -th returned distance usually decreases. A larger value often means that the current result set is still weak. The paper normalizes this score and combines it with a regularization term so that later probes receive a small penalty.

4.2 Calibration

Calibration needs query vectors and exact ground truth. The process is:

1. Train the IVF index.
2. Build an exact flat index for ground truth.
3. Split queries into calibration, tuning, and test sets.
4. For each calibration query and each probe depth, compute the score and the FNR against exact ground truth.
5. Try candidate thresholds.
6. Choose the threshold whose corrected empirical risk is below the requested FNR.

The corrected risk follows the conformal risk control form:

$$\hat{R}_{\text{CRC}} = \frac{m}{m+1} \hat{R} + \frac{1}{m+1}, \quad (5)$$

where m is the number of calibration queries and $\hat{R}$ is the empirical FNR measured after applying a candidate threshold.

4.3 Adaptive Query-Time Search

At query time, ConANN scans centroid lists in increasing centroid distance. After each probe depth, it computes the same score used during calibration. If the score satisfies the calibrated threshold, the search stops and returns the current top- k results. If no threshold accepts the result, the search continues until the maximum probe depth.

```
for each query q:
    compute centroid order for q
    for 0 in 1..N:
        scan the next centroid list
```

```

    update the top-k candidate heap
    score = kth_distance_score(q, current_top_k)
    if score passes calibrated threshold:
        return current_top_k
return current_top_k

```

This works best when it is placed inside the IVF scan loop. If the implementation has to restart search for every probe depth, it repeats distance work and loses much of the expected speed.

4.3.1 Formal Algorithm

Table 1: Algorithm 1: ConANN Calibration Phase (Offline)

Input: error bound α ; total clusters P ; calibration vectors $\{\mathbf{q}_i\}_{i=1}^M$; ground-truth sets $\{S_{\text{GT},i}\}_{i=1}^M$; IVF index.

Output: acceptance threshold $\hat{\lambda}$; RAPS parameters (γ, p_{reg}) .

1. **Build the raw non-conformity matrix.** For each calibration query $i = 1, \dots, M$ and each probe depth $p = 1, \dots, P$:

- (a) scan the IVF index with p probes: $S_{i,p} \leftarrow \text{IVF_scan}(\mathbf{q}_i, p)$;
- (b) compute the normalized score: $\Pi_{i,p} \leftarrow \pi_{\text{norm}}((\mathbf{q}_i, k), S_{i,p})$.

2. **Tune RAPS hyperparameters.** On a held-out tuning split, choose:

$$(\gamma, p_{\text{reg}}) \leftarrow \text{GridSearch}(\Pi, \{S_{\text{GT}}\}, \alpha).$$

3. **Compute regularized scores.** For every calibration query and probe depth:

$$\hat{\Pi}_{i,p} \leftarrow (1 - \Pi_{i,p}) + \gamma \max(0, p - p_{\text{reg}}).$$

4. **Optimize the CRC threshold.** For each candidate λ in a discretized grid over $[0, 1]$:

- (a) for each query, choose the first accepted probe depth:

$$p_i^* \leftarrow \min\{p : \hat{\Pi}_{i,p} \geq \lambda\};$$

- (b) compute the query loss:

$$\text{FNR}_i(\lambda) \leftarrow \text{FNR}(S_{i,p_i^*}, S_{\text{GT},i});$$

- (c) compute corrected empirical risk:

$$\hat{R}(\lambda) \leftarrow \frac{1}{M+1} \left(\sum_{i=1}^M \text{FNR}_i(\lambda) + 1 \right).$$

5. Return the smallest valid threshold:

$$\hat{\lambda} \leftarrow \inf\{\lambda : \hat{R}(\lambda) \leq \alpha\}.$$

5 Part 1: C Implementation

5.1 Goal of the C Prototype

The first part of the project implemented a small ConANN-style IVF system in C. The goal was not to replace FAISS. The goal was to read the paper, rebuild the main search pipeline,

Table 2: Algorithm 2: ConANN Online IVF Search

Input: query $\mathbf{q}$; neighbor count k ; threshold $\hat{\lambda}$; RAPS parameters (γ, p_{reg}) ; IVF index.

Output: approximate nearest-neighbor result $\hat{S}_k$.

1. Initialize $p \leftarrow 0$.
2. Repeat until the threshold accepts the result or all clusters have been considered:
 - (a) increment the probe depth: $p \leftarrow p + 1$;
 - (b) scan the next closest IVF cluster and update the candidate set:
$$S_p \leftarrow \text{IVF_scan}(\mathbf{q}, p);$$
 - (c) compute the normalized k -th-distance score:
$$\pi_p \leftarrow \pi_{\text{norm}}((\mathbf{q}, k), S_p);$$
 - (d) apply the RAPS regularization:
$$\hat{\pi}_p \leftarrow (1 - \pi_p) + \gamma \max(0, p - p_{\text{reg}});$$
 - (e) if $\hat{\pi}_p \geq \hat{\lambda}$, stop and return S_p .
3. If no earlier probe depth is accepted, return the result at the maximum probe depth.

and measure how close a direct C implementation could get to a FAISS CPU baseline.

The prototype includes:

- k-means++ centroid initialization and k-means training,
- assignment of base vectors into inverted lists,
- fixed-probe IVFFlat search,
- exact search for ground truth and metric calculation,
- calibration of adaptive probe thresholds,
- adaptive ConANN-style search,
- benchmark comparison against Python FAISS CPU.

5.2 Code Organization

The C implementation is stored in `part1/Codes`. The main files are:

Table 3: C prototype source files

File	Purpose
<code>fast.h</code>	Shared structs and function declarations
<code>data.c</code>	<code>fvecs/ivecs</code> loading, matrices, metrics
<code>distance.c</code>	L2 distance helpers
<code>kmeans.c</code>	centroid initialization and training
<code>ivf.c</code>	inverted-list build, fixed search, adaptive search
<code>conann.c</code>	calibration cache and threshold selection
<code>bench_sift.c</code>	benchmark driver
<code>bench_faiss_ivf.py</code>	FAISS CPU baseline

5.3 Data Flow

The benchmark driver reads SIFT-format files. The base vectors form the indexed database. The learn vectors train the centroids. Query vectors are split into calibration, validation, and test groups. Ground truth ids are read from the SIFT ground-truth file.

The C index stores each vector id inside the inverted list of its nearest centroid. During fixed IVF search, the code computes distances from the query to all centroids, sorts or selects the closest lists, scans the requested number of lists, and keeps the best k candidates in a heap-like structure.

5.4 Adaptive Search in C

The adaptive part follows the paper at the behavior level. For each target FNR, the prototype runs calibration queries across different probe depths. It records the false negative rate and a distance-based score. The calibration logic then chooses a threshold and a small regularization value. During test search, each query scans lists until the threshold accepts the current result or the maximum probe count is reached.

The implementation also measures the average number of probed lists and the average number of scanned vectors. Those two values explain why a method is fast or slow more directly than recall alone.

5.5 Where the C Prototype Differs From the Paper

The paper implementation modifies FAISS itself. Our Part 1 implementation is a separate C program. This difference matters.

FAISS has optimized distance kernels, tuned memory layout, and efficient code for scanning inverted lists. The C prototype uses simpler data structures and scalar or basic compiler-vectorized loops. It can reproduce the calibration flow, but it cannot match the internal scan efficiency of FAISS without reimplementing a large part of FAISS.

This difference explains the result. The C prototype often probes fewer lists, but FAISS can scan its lists faster.

6 Part 1 Results

6.1 Benchmark Setup

The benchmark used a SIFT-style workload. The quick run used 16 threads. The comparison used three target false negative rates: 0.05, 0.10, and 0.20. For each target, the C adaptive method was compared against a FAISS IVFFlat baseline with a fixed probe count selected to meet or beat the target FNR.

6.2 Measured Results

Table 4: C ConANN prototype against FAISS IVFFlat

Target	C FNR	F FNR	C sec	F sec	C probe	F probe	Ratio
0.05	0.0498	0.0430	0.3452	0.2160	17.93	21.00	0.626
0.10	0.0990	0.0798	0.2152	0.1346	10.42	13.00	0.625
0.20	0.1906	0.1556	0.1410	0.0804	5.95	7.00	0.570

The C implementation stayed close to the requested FNR values. At target 0.05, it measured 0.0498. At target 0.10, it measured 0.0990. At target 0.20, it measured 0.1906. This confirms that the calibration logic worked on the benchmark split.

The runtime result was different. FAISS was faster at all three targets. The speed ratio in the table is FAISS time divided by C time. A value below 1.0 means the C prototype was slower. At target 0.10, the C prototype used approximately 10.42 probes on average while FAISS used 13 probes. Even with fewer probes, the C program took 0.2152 seconds compared with FAISS at 0.1346 seconds.

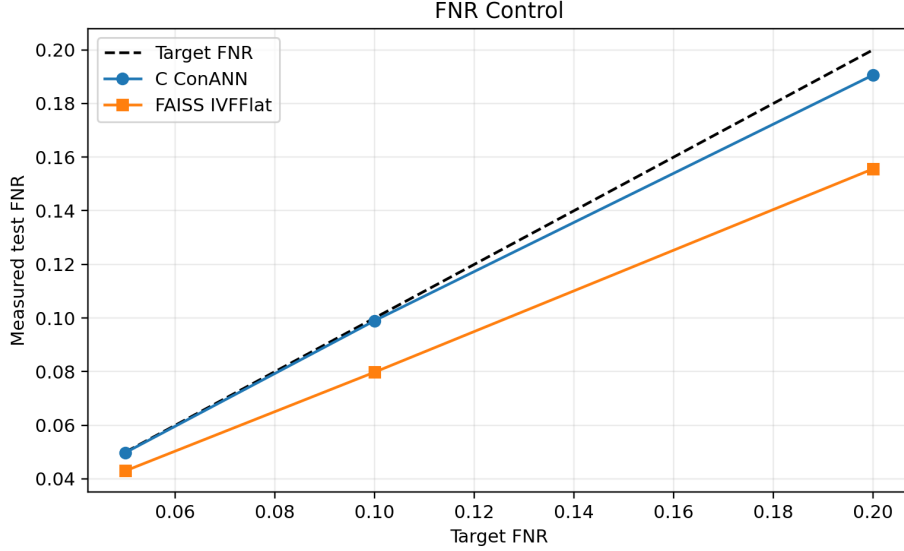


Figure 1: Measured false negative rate compared with requested target.

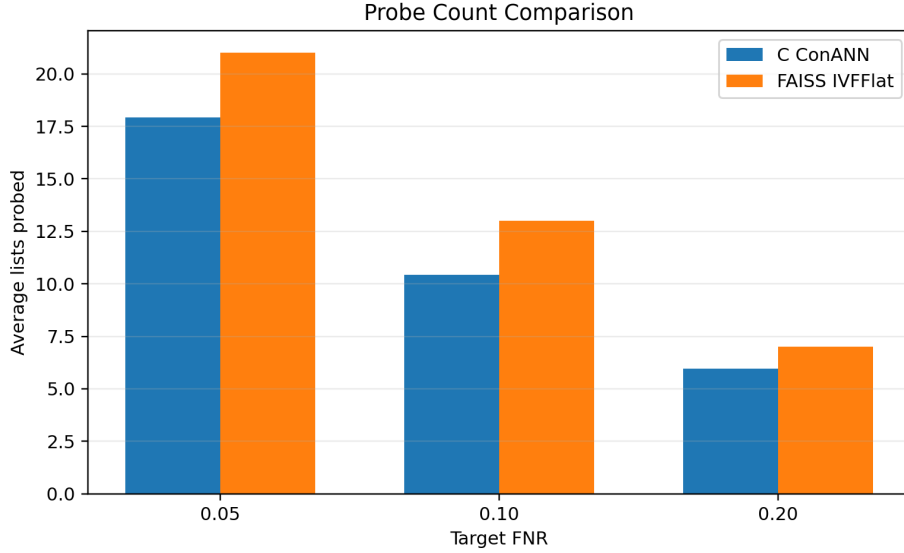


Figure 2: Average probed IVF lists for the C adaptive method and FAISS.

6.3 Additional SIFT10M Result

We also evaluated the ConANN-style stopping rule on a larger SIFT10M run at $k = 10$ with $\gamma = 0.01$ and $c_{reg} = 4$. This run compared the normal ConANN score against our proposed score variant. The proposed variant produced lower empirical FNR and used fewer IVF clusters at every tested target. Its runtime was close to the normal ConANN run, which means the

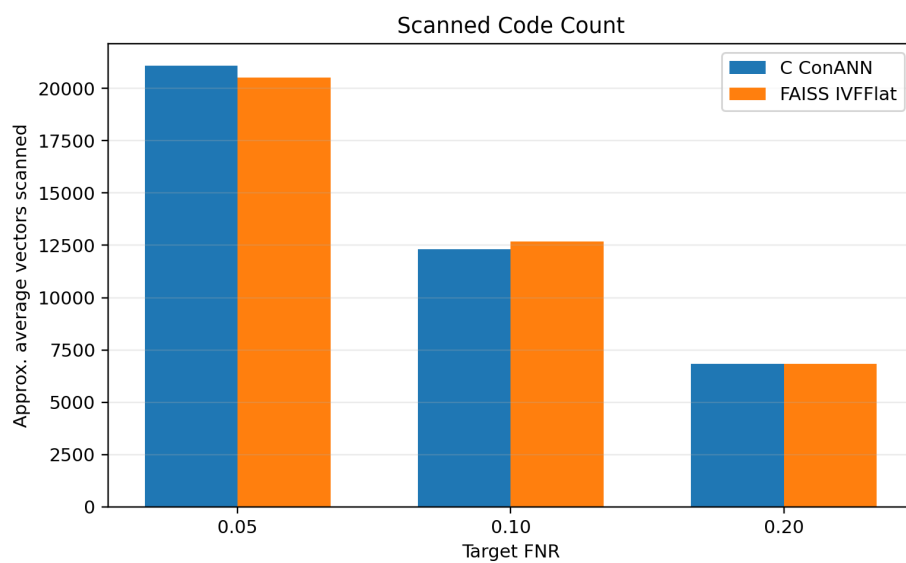


Figure 3: Average scanned vectors for each requested FNR.

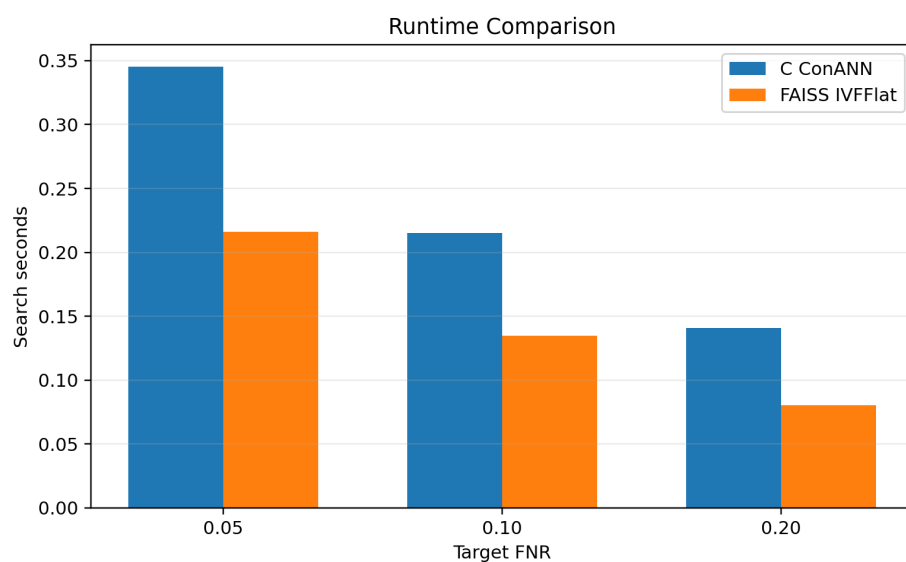


Figure 4: Runtime comparison for the quick SIFT benchmark.

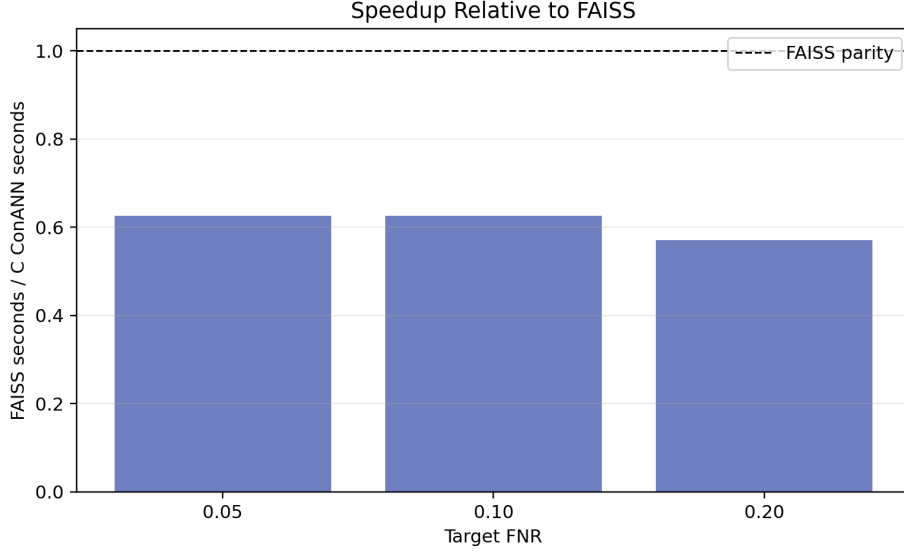


Figure 5: FAISS time divided by C prototype time. Values below one mean FAISS is faster.

score change did not add a large search-time cost in this experiment.

Table 5: SIFT10M comparison between normal ConANN and proposed score variant

Target	N FNR	P FNR	N probe	P probe	N sec	P sec
0.05	0.0409	0.0289	23	20	9.02	9.06
0.10	0.0765	0.0654	18	15	6.84	6.65
0.15	0.1141	0.1041	15	12	5.47	5.26
0.20	0.1626	0.1391	12	10	4.24	4.26
0.25	0.2030	0.1795	10	8	3.53	3.58
0.30	0.2309	0.2082	8	7	2.98	3.51
0.35	0.2764	0.2515	7	5	2.39	2.42
0.40	0.3068	0.2913	6	4	2.09	1.95
0.45	0.3489	0.3142	5	4	1.65	1.70
0.50	0.3828	0.3491	4	3	1.37	1.40

For context, FAISS was also run with fixed `nprobe` values from 1 to 128. In the figures below, the FAISS line uses the smallest fixed `nprobe` that met each target FNR. This is a fair target-based comparison, but it should not be read as the only possible FAISS tuning.

The important point is not only that the proposed variant has lower empirical FNR. Lower FNR can also mean the method is more conservative than necessary. The useful result is that the proposed variant lowered FNR while also reducing average probed clusters, with similar runtime. That is the stronger evidence that the score is improving the stopping decision instead of simply doing more work.

6.4 Additional GIST Result

The same comparison was also run on the GIST dataset at $k = 10$, again using $\gamma = 0.01$ and $c_{reg} = 4$. The baseline ConANN run was compared against the modified fuzzy-clustering variant. On this dataset, the fuzzy variant reduced average probed clusters at every target and usually reduced empirical FNR as well. The biggest change was at strict targets: at target 0.05, the average probe count dropped from 44 to 22.

The FAISS baseline for this pasted GIST run used fixed `nprobe` values from 1 to 40. Its recall improved monotonically from 0.0000 at `nprobe=1` to 0.9256 at `nprobe=40`. That final

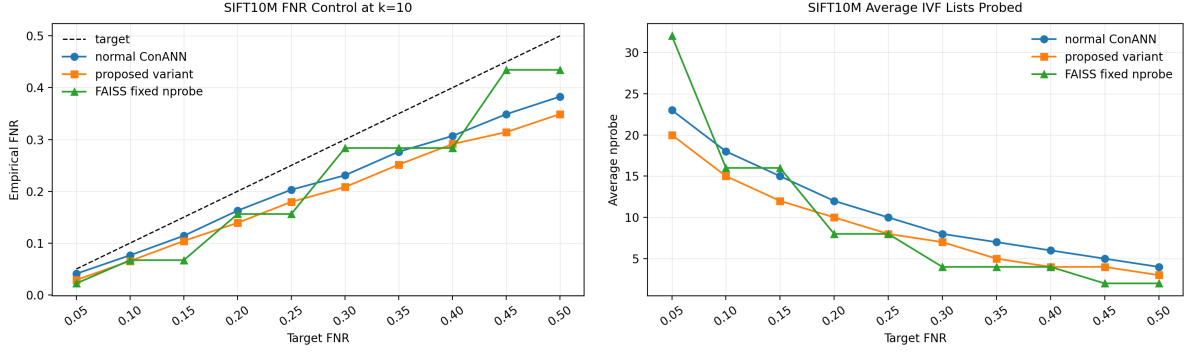


Figure 6: SIFT10M FNR control and average IVF clusters probed.

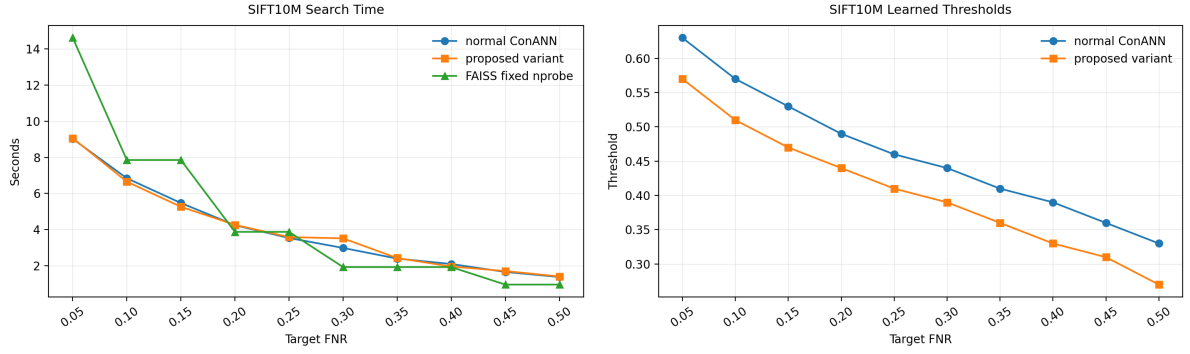


Figure 7: SIFT10M runtime and learned thresholds.

Table 6: GIST comparison between base ConANN and fuzzy-clustering variant

Target	B FNR	Fz FNR	B probe	Fz probe	B sec	Fz sec
0.05	0.0565	0.0547	44	22	8.79	6.13
0.10	0.1094	0.0914	29	19	5.70	5.09
0.15	0.1597	0.1346	23	16	4.25	4.15
0.20	0.2034	0.1903	19	13	3.42	3.41
0.25	0.2477	0.2316	16	11	2.92	2.81
0.30	0.2836	0.2562	14	11	2.80	2.46
0.35	0.3448	0.3137	12	9	2.29	2.07
0.40	0.3657	0.3659	11	8	2.02	1.75
0.45	0.4147	0.3909	9	7	1.60	1.61
0.50	0.4557	0.4556	8	6	1.27	1.31

value is still an FNR of 0.0744, so the plotted FAISS target-matched line has no point at target 0.05. This is reported as missing rather than extrapolated.

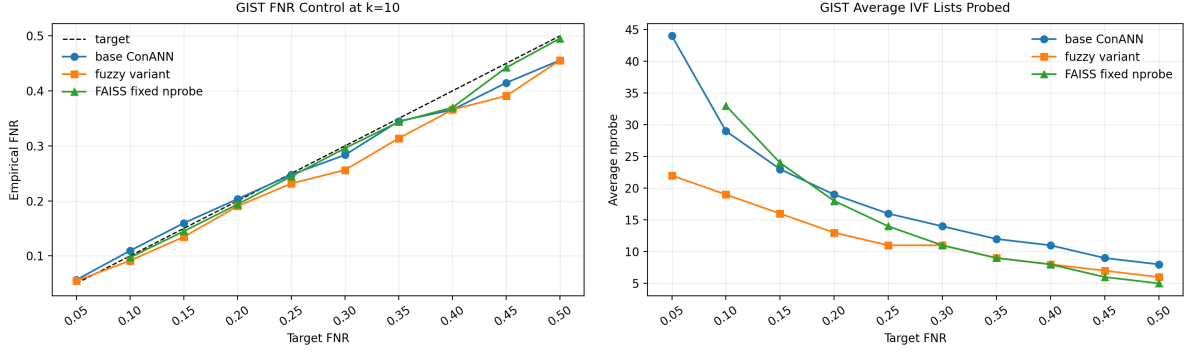


Figure 8: GIST FNR control and average IVF clusters probed.

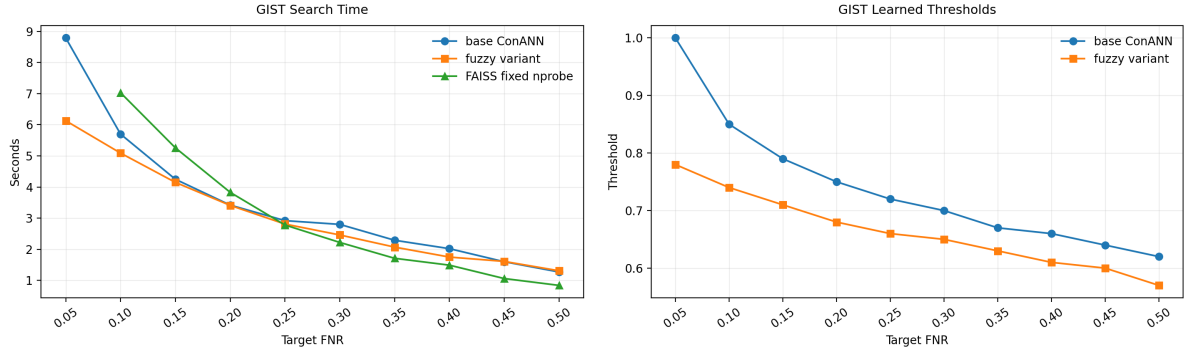


Figure 9: GIST runtime and learned thresholds.

The GIST result supports the same conclusion as the SIFT10M result. The modified score does not merely shift the threshold curve. It changes the stop decision enough to scan fewer clusters while keeping the empirical FNR near the requested target.

6.5 Interpretation

The C prototype answered the first engineering question: the paper method can be implemented in a small codebase and can control FNR on a held-out query split. It also answered the second question: a separate C implementation is not enough to beat FAISS runtime. The scan path matters as much as the stopping rule.

The practical conclusion was to move the adaptive rule closer to FAISS internals. That motivated the second part of the project, where we packaged the ConANN-modified FAISS fork for Python users.

7 Part 2: Python Package

7.1 Goal

The second part of the project made ConANN available as a normal Python package. The package is named `conann`. Users install it with pip and import it with:

```
import conann
```

The package exposes FAISS-style classes and ConANN-specific methods. The main target is CPU IVF search.

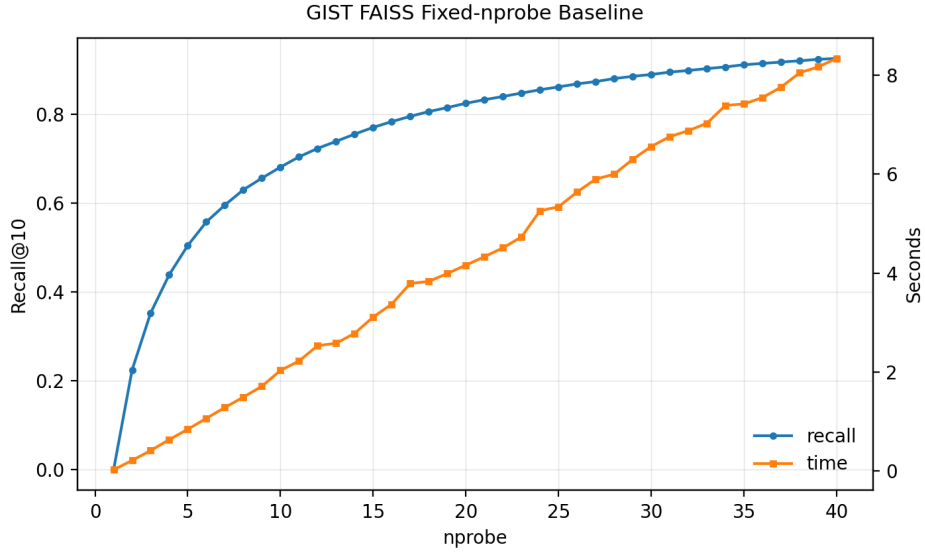


Figure 10: GIST FAISS fixed-nprobe recall and runtime baseline.

7.2 Python API

The package exposes standard index objects such as `IndexFlatL2` and `IndexIVFFlat`. Supported IVF indexes receive these ConANN methods:

- `calibrate_conann(...)`
- `search_conann(...)`
- `evaluate_conann(...)`
- `conann_time_report()`

```
import numpy as np
import conann

d = 32
nlist = 64
k = 10
rng = np.random.default_rng(123)

xb = rng.random((6000, d), dtype=np.float32)
xq = rng.random((240, d), dtype=np.float32)

exact = conann.IndexFlatL2(d)
exact.add(xb)
_, gt = exact.search(xq, k)

quantizer = conann.IndexFlatL2(d)
index = conann.IndexIVFFlat(quantizer, d, nlist, conann.METRIC_L2)
index.train(xb)
index.add(xb)

calibration = index.calibrate_conann(
```

```

    alpha=0.2,
    k=k,
    xq=xq,
    ground_truth=gt,
    calib_sz=0.5,
    tune_sz=0.25,
    dataset_key="example",
)

distances, labels = index.search_conann(xq, calibration, k=k)
metrics = index.evaluate_conann(labels, gt)

```

7.3 Staging the Package

The ConANN fork still builds FAISS-style Python bindings internally. The package staging script rewrites the generated Python files so that the public import name is `conann`. The package does not install a top-level `faiss` alias. This avoids a conflict with the official FAISS package.

The runtime package includes the generated Python wrappers and the native SWIG extension. The metadata in `pyproject.toml` declares:

- package name: `conann`,
- version: `0.1.1`,
- Python requirement: `>=3.10`,
- dependencies: `numpy` and `packaging`.

7.4 Examples

The repository contains example programs for common workflows:

Table 7: Example programs in the Python package

Example	Purpose
<code>01_simple_flat_search.py</code>	exact vector search with NumPy
<code>02_ivf_baseline.py</code>	standard IVF and probe tuning
<code>03_full_conann_workflow.py</code>	calibration, adaptive search, metrics
<code>04_pytorch_embeddings.py</code>	PyTorch tensor embeddings
<code>05_sklearn_text_search.py</code>	TF-IDF/SVD text vectors
<code>06_io_and_metadata.py</code>	index save/load and package metadata

These examples were tested locally with the installed `conann==0.1.1` wheel.

8 Linux Wheel Build

8.1 Why Manylinux Was Needed

The first Linux wheels built on the local machine had the tag `linux_x86_64`. Those wheels worked locally, but PyPI rejected them. Public Linux wheels need a portable platform tag, usually `manylinux`.

The final Linux wheels were built inside a `manylinux_2_28_x86_64` container and repaired with `auditwheel`. The repaired wheels received tags such as:

```
conann-0.1.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
```


8.2 Linux Build Process

The build used Podman with a local reusable builder image. The image was based on the PyPA manylinux image and installed BLAS/LAPACK development packages:

```
FROM quay.io/pypa/manylinux_2_28_x86_64
```

```
RUN dnf install -y openblas-devel lapack-devel && dnf clean all
```

The build script then compiled the ConANN FAISS source for each CPython tag:

```
MANYLINUX_IMAGE=localhost/conann-manylinux:latest \  
bash conann/scripts/build_manylinux_wheels.sh
```

For each Python version, the script:

1. installed Python build dependencies,
2. configured CMake with Python and OpenBLAS paths,
3. built `swigfaiss_avx2`,
4. built `faiss_python_callbacks`,
5. staged generated files into `src/conann`,
6. built a wheel,
7. repaired the wheel with `auditwheel`.

One build issue was Python detection inside CMake. The manylinux Python layout does not always satisfy CMake's full embedded-Python development check. The build was fixed by asking CMake for the interpreter, module development files, and NumPy, instead of requiring the full embedded Python component.

8.3 Linux Wheel Matrix

Table 8: Linux wheels for `conann==0.1.1`

Python	Wheel tag	Status
3.10	cp310-cp310-manylinux	built and checked
3.11	cp311-cp311-manylinux	built and checked
3.12	cp312-cp312-manylinux	built and checked
3.13	cp313-cp313-manylinux	built and checked
3.14	cp314-cp314-manylinux	built and checked

The package was installed from PyPI/TestPyPI in a fresh environment and verified with:

```
python -c "import conann; print(conann.__version__, conann.__faiss_version__)"
```

The expected result is:

```
0.1.1 1.9.0
```

9 Windows Wheel Build

9.1 Windows Requirements

Windows wheels do not need a `manylinux` equivalent. The normal PyPI platform tag is `win_amd64`. The build still needs the Windows native toolchain:

- Visual Studio Build Tools with the C++ workload,
- CMake,
- SWIG,
- vcpkg,
- pyenv-win,
- Python packages: `pip`, `setuptools`, `wheel`, `packaging`, and `numpy`.

The build used `pyenv-win` so that each wheel was compiled with the matching CPython interpreter. For example, Python 3.12 produces a `cp312` wheel.

9.2 Native Dependencies

The Windows build used `vcpkg` for BLAS and LAPACK:

```
openblas:x64-windows  
lapack-reference:x64-windows
```

The CMake configuration disabled GPU and RAFT support and built the CPU AVX2 Python target:

```
-DFAISS_ENABLE_GPU=OFF  
-DFAISS_ENABLE_PYTHON=ON  
-DFAISS_ENABLE_C_API=OFF  
-DFAISS_ENABLE_RAFT=OFF  
-DFAISS_OPT_LEVEL=avx2  
-DBLA_VENDOR=OpenBLAS
```

9.3 MSVC Fixes

The Windows build required small source fixes in the ConANN FAISS fork:

- replace POSIX-only timing headers with standard C++ time utilities,
- add missing standard headers,
- change an OpenMP loop counter from unsigned `size_t` to signed `int64_t`,
- make Python and NumPy discovery run before SWIG target linking.

These changes were build fixes. They did not change the public Python API.

9.4 Windows Wheel Matrix

Runtime smoke tests checked import, IVF creation, calibration, adaptive search, timing reports, and metric evaluation. Metadata checks with `twine check` passed for the final wheels.

Table 9: Windows wheels for <code>conann==0.1.1</code>		
Python	Wheel tag	Status
3.10	cp310-cp310-win_amd64	built and checked
3.11	cp311-cp311-win_amd64	built and checked
3.12	cp312-cp312-win_amd64	built and checked
3.13	cp313-cp313-win_amd64	built and checked
3.14	cp314-cp314-win_amd64	built and checked

10 Verification

10.1 C Prototype Verification

The C code has a small correctness test target:

```
cd part1/Codes
make test
```

The benchmark driver can run without the SIFT files by using synthetic data:

```
./bench_sift --small --threads 4 --no-progress
```

The SIFT quick benchmark writes CSV output and compares with the FAISS baseline:

```
Codes/bench_sift \
--quick \
--threads 16 \
--faiss-csv Results/faiss_cpu_quick.csv \
--csv Results/conann_quick_16t.csv
```

10.2 Python Package Verification

The Python package has import and synthetic tests:

```
python -m unittest discover conann/tests
```

Wheel smoke tests install each wheel in a temporary environment and run a small calibration and adaptive search. The smoke test checks:

- `import conann,`
- package version metadata,
- `IndexFlatL2` and `IndexIVFFlat`,
- ConANN methods on IVF indexes,
- calibration dictionary structure,
- adaptive search output shapes,
- finite returned distances,
- FNR metric range.

The examples in `conann/examples` were also run locally against the installed 0.1.1 wheel.

11 Detailed Design Notes

11.1 Why the Project Was Split Into Two Parts

The project started with the paper rather than the released package. The first question was whether the method could be understood well enough to rebuild its main behavior without using FAISS internals. That is why Part 1 used C. C made memory ownership, list scanning, distance evaluation, and threshold logic easy to inspect. It also made performance problems visible because no hidden library code was doing the expensive work.

The result of Part 1 changed the direction of the second half. The adaptive search rule by itself was not enough. The code also needed the scan path used by FAISS. The paper implementation modified FAISS because FAISS already contains the parts that determine runtime: vector storage, loop structure, SIMD-friendly distance code, heap updates, and thread scheduling. A wrapper that repeatedly calls a public search method cannot reuse the intermediate state inside a scan. It has to redo work. A separate C implementation has the same problem unless it rebuilds those low-level details.

For that reason, Part 2 did not try to keep extending the standalone C program. It moved to the ConANN-modified FAISS C++ source and focused on making that source usable from Python. This was still engineering work, but the goal was different. Instead of writing another search kernel, the package work made the existing C++ implementation installable, importable, and testable across common Python versions and operating systems.

11.2 C Data Structures

The C prototype used simple structures so the search path could be read without following a large framework. The main index state contained the vector dimension, the number of lists, the centroid matrix, and one inverted list per centroid. Each inverted list stored vector ids and vector data. This made list scanning straightforward:

```
for each selected centroid list:
    for each vector id in the list:
        compute L2 distance to the query
        update the top-k result set
```

This layout was good for clarity, but it was not the same as FAISS. FAISS stores inverted-list data in layouts that match its scanner implementations. It also has specialized code paths for different instruction sets. The C prototype used compiler optimization and OpenMP, but it did not implement the same full set of scanner paths.

11.3 Calibration Cache

Calibration can be expensive because it evaluates many query/probe pairs. The C prototype therefore separated score computation from threshold selection. It first built a calibration cache containing, for each calibration query and probe depth, the returned result quality and the score used for stopping. After that, different target FNR values could reuse the same cache.

This design matched the paper’s observation that score computation dominates calibration cost. If scores are already available, selecting another threshold is cheap. The result tables for target FNR values 0.05, 0.10, and 0.20 were computed from the same benchmark setup rather than rebuilding the whole index for each target.

11.4 Why FNR Control Worked But Runtime Did Not

The C prototype’s FNR results were close to the targets because the calibration logic used the same score at calibration time and test time. This consistency is what matters for risk control.

The held-out test split measured whether the threshold chosen on calibration data carried over to unseen queries.

Runtime depends on a different set of facts. A query at target 0.10 scanned about 10.42 lists on average in the C prototype, while the matched FAISS baseline scanned 13 lists. That looks favorable for the C prototype, but the runtime was still slower. FAISS scanned each list faster. The C prototype reduced work at the algorithm level but lost time at the scan-kernel level.

This result is useful because it prevents an inflated claim. The project did not prove that a small C implementation can beat FAISS. It proved that the method needs both parts: calibrated adaptive probing and an optimized IVF scanner.

12 Python Package Engineering

12.1 Import Name and Namespace

The original FAISS Python package uses `import faiss`. Publishing this project under the same import name would confuse users and conflict with the official FAISS package. The package was therefore renamed at the Python distribution level and at the import level:

```
pip install conann
import conann
```

The native SWIG extension still comes from the FAISS build system, but it is staged inside the `conann` package. The staging script rewrites internal imports so that the public module is `conann`. The package exposes `__version__ = "0.1.1"` and `__faiss_version__ = "1.9.0"` so users can see both package version and underlying FAISS version.

12.2 Why Wheels Were Needed

Native C++ packages are difficult for many Python users to install from source. They need a compiler, CMake, SWIG, BLAS/LAPACK, Python headers, and matching NumPy headers. On Windows, they also need Visual Studio Build Tools and vcpkg. On Linux, a local build can create a wheel that works on the same machine but is not accepted by PyPI.

Prebuilt wheels solve this for supported platforms. A user with CPython 3.12 on Linux receives a `cp312 manylinux` wheel. A user with CPython 3.12 on Windows receives a `cp312 win_amd64` wheel. The user does not select the file manually. Pip chooses the wheel whose tag matches the user's Python interpreter and platform.

12.3 Linux Build Details

The Linux build produced five wheels. The final wheel names use the manylinux tags accepted by PyPI:

```
conann-0.1.1-cp310-cp310-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
conann-0.1.1-cp311-cp311-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
conann-0.1.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
conann-0.1.1-cp313-cp313-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
conann-0.1.1-cp314-cp314-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl
```

The local machine first produced plain `linux_x86_64` wheels. PyPI rejected those files because that tag is not a portable public Linux wheel tag. The solution was to build inside the PyPA manylinux container and run `auditwheel repair`. `auditwheel` inspected linked shared

libraries, copied allowed external libraries into the wheel when needed, and rewrote the wheel tag.

One practical problem was dependency installation inside the manylinux container. Installing OpenBLAS and LAPACK through `dnf` took a long time. To avoid repeating that work, a local builder image was created with those packages already installed. The build script then reused that image for every Python version.

12.4 Windows Build Details

Windows did not need a portable container tag. The accepted PyPI tag is `win_amd64`. The harder part was compiling the native extension with MSVC and linking BLAS/LAPACK through `vcpkg`.

The build used `pyenv-win` to install separate CPython versions. Each version compiled its own wheel:

Table 10: Windows Python versions used during the build

Python version	Wheel tag
3.10.11	cp310-cp310-win_amd64
3.11.9	cp311-cp311-win_amd64
3.12.10	cp312-cp312-win_amd64
3.13.13	cp313-cp313-win_amd64
3.14.5	cp314-cp314-win_amd64

The Windows build scripts also had to handle a few practical problems. OneDrive can lock build files, so the build root was moved to a temporary directory. Some shells expose both `Path` and `PATH`, which can confuse MSBuild. The script normalizes the process environment before invoking CMake. Some `pyenv-win` interpreters fail during `ensurepip`; the script supports a mode that skips dependency bootstrapping when the packages are already installed.

12.5 Metadata Repair

The first working Windows wheels installed and ran, but their metadata was not clean enough for PyPI. The long description was malformed, and the description content type was missing. This was fixed by repacking the working native payload through the canonical `pyproject.toml` and README.

After repair, `twine check` passed for all Linux and Windows wheels. The package version was bumped from 0.1.0 to 0.1.1 after the README was rewritten. PyPI does not allow replacing files for an existing version, so a new version was required for the updated project page.

13 How a User Would Use the Package

13.1 Basic Vector Search

A user who already has vectors can use `conann` in the same shape as a basic FAISS script:

```
import numpy as np
import conann

d = 32
xb = np.random.random((5000, d)).astype("float32")
xq = np.random.random((100, d)).astype("float32")
```

```
index = conann.IndexFlatL2(d)
index.add(xb)
distances, labels = index.search(xq, 5)
```

13.2 PyTorch Embeddings

For a PyTorch model, the model output must be detached, moved to CPU, and converted to float32 NumPy data:

```
embeddings = model_output.detach().cpu().numpy().astype("float32")
```

Those arrays can be added to an index. This makes the package usable for small retrieval experiments with sentence models, image models, or custom embedding networks.

13.3 Text Vectors

The example folder also includes a scikit-learn text search script. It builds TF-IDF vectors, reduces them with SVD, normalizes them, and searches them with `conann`. This example is not meant to be a production text retrieval system. It shows that the package accepts ordinary NumPy arrays from another Python library.

14 Comparison With the Paper

The project follows the paper in these parts:

- IVF is the search structure.
- Exact ground truth is used for calibration.
- FNR is the measured risk.
- Calibration chooses thresholds for requested FNR values.
- Search probes lists in centroid order and can stop per query.
- The Python package uses the ConANN-modified FAISS source path.

The project differs from the paper in these parts:

- The standalone C prototype is not the paper's FAISS-integrated scanner.
- The C prototype evaluates a smaller benchmark setting.
- The Python package is CPU-only.
- The package work focuses on usability, wheels, and Python access rather than new search theory.

These differences should be stated when presenting the project. The C prototype shows understanding of the method. The Python package makes the FAISS-integrated implementation easier for Python users to install and test.

15 Limitations

The project has clear limits.

The C prototype is not a FAISS replacement. It implements the method in a small codebase, but it does not use FAISS scan kernels or FAISS memory layout. Because of that, it controlled FNR but did not beat FAISS runtime.

The Python package is CPU-only. It focuses on the ConANN-enabled IVF path. It is not a full replacement for all FAISS features, and it does not expose GPU FAISS support.

The calibration guarantee depends on the calibration queries matching the distribution of future queries. If the query distribution changes after calibration, the measured FNR can move away from the target.

16 Conclusion

This project produced two connected results. The C prototype showed how ConANN’s calibration and adaptive probing can be implemented from the paper. It also showed why speed depends on running the stopping rule inside optimized IVF internals. On the SIFT quick benchmark, the prototype measured FNR values close to the requested targets: 0.0498, 0.0990, and 0.1906. It did not beat FAISS runtime.

The Python package addressed the distribution problem. It packaged the ConANN-modified FAISS CPU code as `conann`, built wheels for Linux and Windows, supported CPython 3.10 through 3.14, and exposed ConANN methods through Python. A user can now install the package with pip and run ConANN-style IVF search without building the research fork manually.

References

- [1] Sonia Horchidan, Fabian Zeiher, Henrik Bostrom, and Paris Carbone. *ConANN: Conformal Approximate Nearest Neighbor Search*. Proceedings of the VLDB Endowment, 19(1):29–42, 2025. DOI: 10.14778/3772181.3772184.
- [2] Jeff Johnson, Matthijs Douze, and Herve Jegou. *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data, 7(3):535–547, 2021.
- [3] Stephen Bates, Anastasios Angelopoulos, Lihua Lei, Jitendra Malik, and Michael Jordan. *Distribution-free, risk-controlling prediction sets*. Journal of the ACM, 68(6), 2021.
- [4] Anastasios Angelopoulos, Stephen Bates, Jitendra Malik, and Michael Jordan. *Uncertainty sets for image classifiers using conformal prediction*. International Conference on Learning Representations, 2021.
- [5] Herve Jegou, Matthijs Douze, and Cordelia Schmid. *Product quantization for nearest neighbor search*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 33(1):117–128, 2011.